



UNIVERSITÀ DEGLI STUDI
DI SALERNO

Corso di Fondamenti di Intelligenza Artificiale

RICONOSCIMENTO AUTOMATICO DELLA LIS



Repository Github:

https://github.com/Swami-Falasca/LIS_AI.git

Dima Isabella (0512119719)

Falasca Swami (0512119323)

Tirelli Giuseppe (0512120367)

Indice

1	Introduzione	1
2	Descrizione dell'agente	1
2.1	Obiettivi	1
2.2	Specifica PEAS	1
2.2.1	Caratteristiche dell'ambiente	2
2.3	Analisi del Problema	2
2.3.1	Formalizzazione del problema	3
3	Raccolta e preprocessing dei dati	3
3.1	Scelta del dataset	3
3.2	Analisi del dataset	4
3.3	Operazioni di preprocessing	4
3.3.1	Riduzione delle dimensioni del problema	4
3.3.2	Tecniche di smoothing e normalizzazione	5
3.3.3	Data augmentation (flipping)	5
4	Modelli di Classificazione- Lettere Statiche	5
4.1	Random Forest Classifier (RFC)	5
4.2	Support Vector Classifier (SVC)	6
4.3	K-Nearest Neighbours KNN	6
4.4	Scelta finale	7
5	Modelli di Classificazione -Lettere Dinamiche	7
5.1	Long short-term memory (LSTM)	7
5.2	Gated Recurrent Unit (GRU)	7
5.3	Scelta finale	8
6	Librerie	8
7	Glossario	8

1 Introduzione

La Lingua dei Segni Italiana (LIS) è una lingua visivo-gestuale e rappresenta un linguaggio vero e proprio, con proprie regole grammaticali ed è riconosciuta ufficialmente dallo Stato. E' composta da segni basati sul movimento delle mani uniti alle espressioni facciali e postura consentendo una comunicazione che non sia basata sulla lingua parlata.

La LIS è una lingua che è stata approvata in Italia qualche anno fa, quindi ad oggi i sistemi che aiutano a tradurre i segni sono pochi a differenza dell'ASL (American Sign Language) che risulta essere più diffusa. Inoltre la LIS ha differenti sfumature per quanto riguarda i gesti perchè molti variano in base alla regione in quanto vengono utilizzati molto i dialetti.

La popolazione risulta essere poco informata sulla LIS e il nostro obiettivo è quello di abbattere le barriere di comunicazione con le persone sorde.

Il progetto **LIS_AI** propone un sistema di riconoscimento automatico della LIS per contrastare questa disinformazione della popolazione italiana.

E' basato su classificazione dei modelli per tradurre gesti in tempo reale.

L'obiettivo è facilitare la comunicazione tra persone sorde e persone udenti senza richiedere conoscenze della LIS, per favorire inclusività delle persone sorde in vari contesti come lavorativo, scolastico.

2 Descrizione dell'agente

2.1 Obiettivi

Lo scopo di questo progetto è realizzare un agente intelligente in grado di riconoscere gesti della Lingua dei Segni Italiana (LIS) da immagini e tradurli in testo. L'agente deve essere in grado di:

1. Riconoscere i gesti LIS, fondamentale per comunicare in modo efficace con le persone sorde
2. Tradurre i gesti riconosciuti in testo in tempo reale
3. Mettere a disposizione un'interfaccia semplice per l'utilizzo del sistema

2.2 Specifica PEAS

Performance	Accuratezza: Corretta classificazione dei gesti LIS Tempo di risposta: Traduzione in tempo reale Robustezza: Diverse condizioni di illuminazione
Environment	Input: Video acquisiti via webcam Posizione: Utente e sfondo variabile Illuminazione: Non controllata Distrazioni: Potenziale presenza di elementi distraenti
Actuators	Output: Visualizzazione della lettera tradotta Interfaccia: Comandi avvio/arresto, salvataggio, pulizia cronologia Feedback: Indicazione visiva della confidenza
Sensors	Webcam: Acquisizione video Frame: Elaborazione dei frame Features: Estrazione delle caratteristiche visive

2.2.1 Caratteristiche dell'ambiente

- **Parzialmente osservabile:** non tutti i dettagli dei gesti sono sempre visibili
- **Dinamico:** l'utente si muove liberamente durante l'esecuzione dei gesti
- **Sequenziale:** i gesti hanno una dimensione temporale da considerare
- **Discreto:** insieme finito di gesti da riconoscere

2.3 Analisi del Problema

La sfida del riconoscimento della Lingua Italiana dei Segni è stata affrontata come un problema di classificazione. I gesti sono stati infatti suddivisi ed etichettati per lettera rappresentata. Tuttavia, nella progettazione sono sorti numerosi problemi di grande importanza per la buona riuscita del progetto:

- I segni LIS sono veicolati con la mano dominante. La mano utilizzata deve essere quindi influente nel riconoscimento del segno.

- Come rappresentare al meglio i dati in modo da estrarre solo le informazioni essenziali?
- Come scegliere un formato adatto, ponderando anche l'alleggerimento del carico computazionale e l'ottimizzazione della performance?
- Non tutte le lettere della LIS sono riconosciute allo stesso modo. Ad esempio, la lettera "S" richiede un movimento della mano. Potrebbe essere opportuno utilizzare più di un modello di classificazione.

2.3.1 Formalizzazione del problema

- **Stato iniziale:** Frame video non processato dalla webcam
- **Azioni possibili:**
 - Acquisire nuovo frame dalla webcam
 - Classificare il gesto utilizzando il modello addestrato
 - Visualizzare il risultato della classificazione
- **Test obiettivo:** Riconoscimento corretto del gesto eseguito
- **Costo del cammino:** Errore di classificazione e tempo di computazione

La complessità del problema richiede l'esplorazione di multiple architetture neurali per identificare il miglior compromesso tra accuratezza e efficienza computazionale.

Il miglior compromesso tra accuratezza ed efficienza è fondamentale. Per trovare ciò ci dobbiamo basare sulla velocità con cui otteniamo i risultati e l'accuratezza dei risultati ottenuti.

3 Raccolta e preprocessing dei dati

3.1 Scelta del dataset

Il dataset scelto per questo progetto è il **LIS Dataset** disponibile su Kaggle, contenente oltre 6000 immagini in formato JPG delle lettere dell'alfabeto italiano della Lingua dei Segni Italiana opportunamente etichettate. Sono presenti tutte le lettere dell'alfabeto italiano ad eccezione delle seguenti: G, J, S e Z perchè richiedono movimenti più complessi.

La scelta è ricaduta su questo dataset per vari motivi:

- Rappresenta solo la LIS e non l'ASL
- È di dimensioni gestibili per le risorse computazionali disponibili

Nonostante l'ottimo dataset, non possiamo ignorare il fatto che è, purtroppo, **incompleto**. Non essendoci su Kaggle o piattaforme simili alternative valide, abbiamo optato per registrare noi stessi le lettere rimanenti (record_dynamic.py), ottenendo così 100 campioni di ogni lettera. Le modalità di registrazione delle lettere mancanti verranno approfondite nella sezione 3.3.1.

Per maggiore chiarezza, da adesso in poi ci riferiremo a tutte le lettere presenti nel dataset originale, ovvero rappresentabili con una semplice immagine statica, come **lettere statiche**. Le lettere non rappresentabili staticamente, ovvero richiedenti un movimento, saranno definite **lettere dinamiche**.

3.2 Analisi del dataset

Il dataset originale presenta le seguenti caratteristiche:

- Numero di classi: 6428 foto LIS
- Formato: foto in JPG
- Risoluzione: variabile tra le foto
- Sfondo: non uniforme tra i diversi campioni

3.3 Operazioni di preprocessing

Per preparare i dati all'addestramento dei modelli, sono state effettuate le seguenti operazioni:

3.3.1 Riduzione delle dimensioni del problema

Per gestire al meglio le informazioni del dataset, eliminando i dettagli non essenziali e focalizzando l'attenzione sui contenuti rilevanti abbiamo optato per una riduzione delle dimensioni dei dati.

In particolare, sfruttando la libreria `mediapipe`, abbiamo identificato sulle mani 21 landmarks (punti chiave anatomici), corrispondenti a polso (1 punto) e giunture delle dita (4 punti per dito = 20 punti). I landmarks connessi tra loro che sono stati rappresentati come vettori di coordinate (x,y), per un totale di 42 features per immagine.

La struttura dati contenente l'insieme di tutti i vettori di landmarks etichettati è stata serializzata in formato `pickle`, particolarmente adatto al nostro problema per il suo supporto nativo per array `Numpy` e gli oggetti `scikit-learn`. Questo ci consente di lavorare con un dataset ridotto alle informazioni essenziali, riducendo drasticamente il carico computazionale. Da diverse centinaia di MegaByte di immagini `jpg` si passa ad un singolo file di meno di 10 MegaByte.

Le registrazioni delle lettere dinamiche (J, Z, S, G) abbiamo adottato una strategia analoga. Per semplicità, abbiamo saltato il passaggio da formato video a immagini statiche e registrato direttamente i vettori di landmarks per ogni frame dei 30 totali per ogni acquisizione. Le registrazioni sono state effettuate eseguendo il file `record_dynamic.py`, anch'esso disponibile nel repository GitHub del progetto. In totale, sono state acquisite 100 registrazioni per ogni lettera dinamica

3.3.2 Tecniche di smoothing e normalizzazione

Per quanto riguarda le lettere statiche abbiamo implementato una normalizzazione spaziale. Per rendere i gesti invarianti alla posizione della mano nell'immagine, abbiamo centrato le coordinate dei landmarks rispetto al polso.

Per le lettere dinamiche, essendo composte da una sequenza di frame, abbiamo dovuto seguire un approccio diverso. I gesti sono stati infatti normalizzati e scalati rispetto ai minimi e ai massimi tenendo conto di tutta la traiettoria, e non solo di un singolo frame. Abbiamo poi applicato dei filtri per ridurre il rumore all'interno del dataset, filtrando le fluttuazioni rapide ed eliminando così gli outlier dovuti a registrazioni imperfette.

3.3.3 Data augmentation (flipping)

Come spiegato nella sezione 2.3, la mano utilizzata nei segni deve essere ininfluyente per il riconoscimento.

Per evitare bias di lateralità (ovvero la tendenza del modello a riconoscere più facilmente gesti di una mano piuttosto che dell'altra) e aumentare la robustezza del modello, abbiamo effettuato un'operazione di data augmentation tramite "flipping". Questa tecnica ci permette di simulare la mano opposta senza l'inserimento di dati aggiuntivi.

Nel dettaglio, dopo aver estratto i landmarks in ogni immagine, abbiamo invertito le coordinate orizzontali (intuitivamente abbiamo, quindi, capovolto le mani) e aggiunto i vettori "flipped" al dataset di landmarks. Così facendo, il dataset finale è raddoppiato e il modello finale sarà invariante alla lateralità, simulando così con maggiore efficacia la simmetria intrinseca di molti gesti della LIS.

4 Modelli di Classificazione- Lettere Statiche

Considerata la natura diversa di lettere statiche e dinamiche, abbiamo deciso di inserire nel nostro progetto due modelli di classificazione specializzati sull'aspetto statico o dinamico. Questa differenza sarà approfondita nel punto 5. Il risultato finale dell'inferenza sarà dettato dalla confidence dei modelli sulla predizione.

4.1 Random Forest Classifier (RFC)

La nostra prima scelta per il modello di classificazione delle lettere statiche è stata Random Forest Classifier (RF), disponibile nella libreria Scikit-Learn. La sua semplicità e velocità (e quindi bassa latenza) ci sono sembrate un buon punto di partenza per la scelta dell'algoritmo. Nel dettaglio, RF è un algoritmo che combina una moltitudine di alberi decisionali indipendenti. L'output finale della classificazione è la classe selezionata dal maggior numero di alberi. Ogni albero viene addestrato su un set di dati casuali. Nel nostro caso, il test set è formato dal 20% dei dati totali, opportunamente mischiati (shuffled) e stratificati (sono state mantenute le proporzioni per ogni label).

Il modello allenato sembrava molto promettente, avendo ottenuto un punteggio percentuale di Accuracy del 99,76%. Tuttavia, al momento del test di inferenza in tempo reale con

webcam, abbiamo riscontrato difficoltà del modello nel riconoscimento accurato di gesti simili e un’instabilità generale (jittering), il che andava in contrasto con l’altissima precisione calcolata nel testing.

L’elevata Accuracy si è quindi rivelata fuorviante e abbiamo stabilito che fosse un probabile sintomo di overfitting. Il modello non apprendeva la reale configurazione dei gesti e generava infatti risultati non accurati alla minima variazione rispetto ai dati di training. Abbiamo quindi deciso di espandere le possibilità di scelta, cercando un modello meno rigido e più resistente a **overfitting**.

4.2 Support Vector Classifier (SVC)

Support Vector Classifier (SVC) è un algoritmo che trova l’iperpiano ottimale per la separazione di diverse classi. Usa il “**kernel trick**”, una funzione che mappa i dati ad uno spazio iperdimensionale, permettendo all’algoritmo di gestire dati non linearmente separabili.

Ci siamo indirizzati su questo algoritmo proprio per l’utilizzo dei margini: abbiamo pensato infatti che potesse essere più robusto a piccoli rumori rispetto a un albero decisionale. Dopo aver allenato il modello, siamo stati sorpresi nel vedere un’Accuracy molto inferiore al modello precedente: 85.53%.

Ciononostante, abbiamo deciso ugualmente di effettuare test empirici. I risultati sono stati molto positivi: in sede di test real-time, SVC ha dimostrato una capacità di generalizzazione e una stabilità decisamente superiore a RF, che soffriva di overfitting. L’intuizione dei margini si è rivelata vincente.

La minore accuratezza in fase di test è probabilmente dovuta al fatto che SVC ha preferito non memorizzare il rumore presente nel dataset (differenza di RF che lo ha “imparato a memoria”, arrivando al 99%), accettando un bias leggermente superiore in cambio di una varianza (minore jittering) molto inferiore durante il test in real-time. Tuttavia, per un ulteriore metro di paragone, abbiamo deciso di testare un ultimo algoritmo, il cui funzionamento è radicalmente diverso dai precedenti.

4.3 K-Nearest Neighbours KNN

K-Nearest Neighbors (KNN), a differenza dei precedenti algoritmi, non crea un modello durante l’allenamento (lazy learner) bensì basa la sua classificazione in base alle classi dominanti e più vicine, ovvero agli esempi più simili presenti nel dataset.

Nonostante l’ottima Accuracy misurata (98,84%), le previsioni real-time si sono rivelate piuttosto deludenti. Una minima variazione nell’inclinazione della mano era sufficiente per confondere il modello e generare previsioni erranee. Ad esempio, la lettera “N” (rappresentata da indice e medio verso il basso) alla minima rotazione veniva sistematicamente confusa con la lettera “M” (quattro dita verso il basso): il modello, quindi, non sapeva distinguere la struttura anatomica della mano.

4.4 Scelta finale

Confrontando i tre algoritmi testati, abbiamo concluso senza ombra di dubbio che il modello migliore per le lettere statiche fosse il Support Vector Classifier. Nonostante le Accuracy più alte vantate dagli altri modelli, nel mondo reale SVC è più stabile, più accurato e meno soggetto a overfitting.

Altre metriche, come il tempo di allenamento e punteggio di Recall, sono disponibili nella tabella sottostante.

Modello	Accuracy (%)	Precision	Recall	F1-Score	Tempo Addestramento (s)
Random Forest	99.76	0.9976	0.9976	0.9976	17.682
SVC	85.53	0.8655	0.8553	0.8448	6.518
K-Neighbors	98.84	0.9886	0.9884	0.9884	/

Nota su KNN: il tempo di allenamento è nullo in quanto si tratta di un algoritmo lazy learner.

5 Modelli di Classificazione -Lettere Dinamiche

Come menzionato sopra, abbiamo deciso di utilizzare due modelli diversi per le lettere statiche e dinamiche. Il motivo è un limite concettuale: i modelli sopra citati considererebbero i frame come indipendenti e non come una sequenza, perdendo quindi il movimento.

Per il nostro problema è necessario un modello che memorizzi i frame in sequenza e che ricordi tutte le fasi del gesto.

5.1 Long short-term memory (LSTM)

LSTM immagazzina i dati all'interno di una rete neurale, che permette di accedere a informazioni precedenti e di usare queste ultime nei calcoli successivi.

Questa soluzione è perfetta per memorizzare dati sequenziali come dei frame. Nello specifico, le LSTM sono dotate di tre “porte” (gates) che regolano il flusso di informazioni: decidono cosa dimenticare, cosa memorizzare e cosa trasmettere al calcolo successivo. Questo permette al modello di riconoscere un gesto dinamico (come la lettera 'J') dall'intero movimento. Il modello ha registrato un'accuratezza del 99.38%.

5.2 Gated Recurrent Unit (GRU)

Parallelamente alla LSTM, abbiamo testato la Gated Recurrent Unit (GRU). In maniera simile a LSTM, utilizza dei meccanismi di gating per controllare il flusso di informazioni, con la differenza che utilizza meno parametri ed è quindi meno complesso. L'accuratezza misurata è stata del 100%.

5.3 Scelta finale

In fase di testing, sia analitico sia empirico, abbiamo riscontrato che GRU fosse la scelta migliore per il nostro progetto, nonostante i due modelli si comportassero in maniera simile.

L'accuratezza non è stata l'unico fattore discriminante, in quanto eccellente in entrambi i casi. Ciò che ci ha portati a prendere questa decisione è stata principalmente la migliore precision in testing real-time dovuta all'architettura semplificata di GRU.

6 Librerie

Le librerie che sono state usate per sviluppare il sistema sono:

- ◇ mediapipe: serve ad elaborare immagini e video.
- ◇ matplotlib.pyplot: serve per creare visualizzazioni interattive, e per creare grafici.
- ◇ pickle: serializzazione strutture dati e modelli addestrati
- ◇ numpy: gestione di array

Per i modelli statici:

- ◇ Scikit-Learn: usata per i classificatori e le metriche

Per i modelli dinamici:

- ◇ keras: addestramento di reti neurali

7 Glossario

Landmarks i punti chiave anatomici corrispondenti al polso (1 punto) e giunture delle dita (4 per dito), per un totale di 21 landmarks

Flipping tecnica di inversione orizzontale del landmarks.

Smoothing tecnica di filtraggio per ridurre rumore e outlier.

Frame singola immagine in una sequenza video.

Classificazione problema di apprendimento automatico supervisionato.

Dataset insieme di dati usati per addestrare e testare il modello.

Bias di lateralità tendenza del modello a riconoscere più accuratamente i gesti di una mano rispetto ad un'altra.

Overfitting (Sovradattamento) quando un modello si adatta troppo fedelmente ai dati e generalizza male, imparando "a memoria".

Ensambe Learning tecnica di machine learning che combina più modelli per migliorare la performance predittiva.

Lazy Learner algoritmo di apprendimento il cui modello viene costruito al momento della previsione, ovvero rimanda i calcoli delle previsioni finché non è necessario.

Iperpiano il piano che massimizza la distanza tra i punti delle classi nei problemi di ottimizzazione

jitting fenomeno di instabilità nella predizione del modello.